

```
In [11]: import os
import cv2
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.models import Model, Sequential
import ssl
ssl._create_default_https_context = ssl._create_unverified_context
```

```
In [2]: spiral_dir = r"C:\Users\Pritam\Spiral_HandPD"
meander_dir = r"C:\Users\Pritam\Meander_HandPD"

IMG_SIZE = 224
```

```
In [3]: data = []
labels = []

def load_images(folder, label):
    for img_name in os.listdir(folder):
        if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):
            img_path = os.path.join(folder, img_name)
            img = cv2.imread(img_path)

            if img is None:
                continue

            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            img = img.astype("float32") / 255.0

            data.append(img)
            labels.append(label)
```

```
In [4]: # Spiral
load_images(os.path.join(spiral_dir, "healthy"), 0)
load_images(os.path.join(spiral_dir, "parkinson"), 1)

# Meander
load_images(os.path.join(meander_dir, "healthy"), 0)
load_images(os.path.join(meander_dir, "parkinson"), 1)

X = np.array(data, dtype=np.float32)
y = np.array(labels)

print("Total images:", X.shape)
print("Labels distribution:", np.unique(y, return_counts=True))
```

Total images: (808, 224, 224, 3)
Labels distribution: (array([0, 1]), array([216, 592]))

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train:", X_train.shape)
print("Test :", X_test.shape)
```

Train: (646, 224, 224, 3)
Test : (162, 224, 224, 3)

```
In [6]: datagen = tf.keras.preprocessing.image.ImageDataGenerator(
    rotation_range=15,
    zoom_range=0.15,
    horizontal_flip=True,
```

```

        fill_mode="nearest"
    )

    datagen.fit(X_train)

```

```

In [20]: from sklearn.utils.class_weight import compute_class_weight
import numpy as np

```

```

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

class_weights_dict = dict(enumerate(class_weights))
print("Class weights:", class_weights_dict)

```

Class weights: {0: np.float64(1.8670520231213872), 1: np.float64(0.6828752642706131)}

```

In [8]: !pip install vit-keras

```

```

Collecting vit-keras
  Downloading vit_keras-0.2.0-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: scipy in c:\users\pritam\miniconda3\envs\data_analysis_env\lib\site-packages
(from vit-keras) (1.15.3)
Collecting validators (from vit-keras)
  Downloading validators-0.35.0-py3-none-any.whl.metadata (3.9 kB)
Requirement already satisfied: numpy<2.5,>=1.23.5 in c:\users\pritam\miniconda3\envs\data_analysis_env\lib\si
te-packages (from scipy->vit-keras) (2.0.1)
Downloading vit_keras-0.2.0-py3-none-any.whl (24 kB)
Downloading validators-0.35.0-py3-none-any.whl (44 kB)
Installing collected packages: validators, vit-keras

----- 0/2 [validators]
----- 0/2 [validators]
----- 2/2 [vit-keras]

```

Successfully installed validators-0.35.0 vit-keras-0.2.0

```

In [7]: from vit_keras import vit
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

```

```

In [8]: import certifi
print(certifi.where())

```

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\certifi\cacert.pem

```

In [12]: inputs = Input(shape=(224, 224, 3))

vit_base = vit.vit_b16(
    image_size=224,
    pretrained=True,
    include_top=False,
    pretrained_top=False
)

# Freeze ViT Layers (important for small dataset)
for layer in vit_base.layers:
    layer.trainable = False

x = vit_base(inputs)
x = BatchNormalization()(x)

```

Downloading data from https://github.com/faustomorales/vit-keras/releases/download/dl/ViT-B_16_imagenet21k+imagenet2012.npz

347502902/347502902 ————— 50s 0us/step

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\vit_keras\utils.py:85: UserWarning: Resizing position embeddings from 24, 24 to 14, 14
warnings.warn(

```

In [13]: x = Dense(526, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)

x = Dense(256, activation='relu')(x)

```

```
x = BatchNormalization()(x)
x = Dropout(0.3)(x)

x = Dense(128, activation='relu')(x)
x = BatchNormalization()(x)

x = Dense(64, activation='relu')(x)
x = BatchNormalization()(x)

x = Dense(32, activation='relu')(x)
x = BatchNormalization()(x)

x = Dense(16, activation='relu')(x)
x = BatchNormalization()(x)

x = Dense(8, activation='relu')(x)
x = BatchNormalization()(x)

x = Dense(4, activation='relu')(x)
x = Dense(2, activation='relu')(x)

outputs = Dense(1, activation='sigmoid')(x) # Binary output
```

```
In [17]: model = Model(inputs=inputs, outputs=outputs, name="Custom_ViT_526_to_1")
          model.summary()
```

Model: "Custom_ViT_526_to_1"

Layer (type)	Output Shape	Param #
input_layer_28 (InputLayer)	(None, 224, 224, 3)	0
vit-b16 (Functional)	(None, 768)	85,798,656
batch_normalization (BatchNormalization)	(None, 768)	3,072
dense (Dense)	(None, 526)	404,494
batch_normalization_1 (BatchNormalization)	(None, 526)	2,104
dropout_108 (Dropout)	(None, 526)	0
dense_1 (Dense)	(None, 256)	134,912
batch_normalization_2 (BatchNormalization)	(None, 256)	1,024
dropout_109 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dense_3 (Dense)	(None, 64)	8,256
batch_normalization_4 (BatchNormalization)	(None, 64)	256
dense_4 (Dense)	(None, 32)	2,080
batch_normalization_5 (BatchNormalization)	(None, 32)	128
dense_5 (Dense)	(None, 16)	528
batch_normalization_6 (BatchNormalization)	(None, 16)	64
dense_6 (Dense)	(None, 8)	136
batch_normalization_7 (BatchNormalization)	(None, 8)	32
dense_7 (Dense)	(None, 4)	36
dense_8 (Dense)	(None, 2)	10
dense_9 (Dense)	(None, 1)	3

Total params: 86,389,199 (329.55 MB)

Trainable params: 586,947 (2.24 MB)

Non-trainable params: 85,802,252 (327.31 MB)

```
In [18]: model.compile(
          optimizer=Adam(learning_rate=1e-4),
          loss='binary_crossentropy',
          metrics=['accuracy']
        )
```

```
In [21]: history = model.fit(
          datagen.flow(X_train, y_train, batch_size=16),
          validation_data=(X_test, y_test),
          epochs=25,
          class_weight=class_weights_dict,
          verbose=1
        )
```

```
C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignored.  
    self._warn_if_super_not_called()
```

Epoch 1/25
41/41  550s 13s/step - accuracy: 0.6687 - loss: 0.6878 - val_accuracy: 0.7469 - val_loss: 0.6606

Epoch 2/25
41/41  526s 13s/step - accuracy: 0.6548 - loss: 0.6856 - val_accuracy: 0.7531 - val_loss: 0.6615

Epoch 3/25
41/41  544s 13s/step - accuracy: 0.6502 - loss: 0.6841 - val_accuracy: 0.7531 - val_loss: 0.6582

Epoch 4/25
41/41  541s 13s/step - accuracy: 0.6331 - loss: 0.6956 - val_accuracy: 0.7284 - val_loss: 0.6616

Epoch 5/25
41/41  541s 13s/step - accuracy: 0.6703 - loss: 0.6868 - val_accuracy: 0.7469 - val_loss: 0.6611

Epoch 6/25
41/41  586s 14s/step - accuracy: 0.7012 - loss: 0.6776 - val_accuracy: 0.7222 - val_loss: 0.6595

Epoch 7/25
41/41  552s 14s/step - accuracy: 0.7167 - loss: 0.6736 - val_accuracy: 0.7099 - val_loss: 0.6616

Epoch 8/25
41/41  533s 13s/step - accuracy: 0.7059 - loss: 0.6703 - val_accuracy: 0.7407 - val_loss: 0.6642

Epoch 9/25
41/41  522s 13s/step - accuracy: 0.6966 - loss: 0.6720 - val_accuracy: 0.7037 - val_loss: 0.6688

Epoch 10/25
41/41  523s 13s/step - accuracy: 0.7214 - loss: 0.6701 - val_accuracy: 0.7469 - val_loss: 0.6623

Epoch 11/25
41/41  591s 14s/step - accuracy: 0.7043 - loss: 0.6762 - val_accuracy: 0.7099 - val_loss: 0.6583

Epoch 12/25
41/41  531s 13s/step - accuracy: 0.7291 - loss: 0.6619 - val_accuracy: 0.7407 - val_loss: 0.6538

Epoch 13/25
41/41  532s 13s/step - accuracy: 0.7043 - loss: 0.6739 - val_accuracy: 0.7407 - val_loss: 0.6594

Epoch 14/25
41/41  557s 13s/step - accuracy: 0.7074 - loss: 0.6593 - val_accuracy: 0.7469 - val_loss: 0.6587

Epoch 15/25
41/41  562s 14s/step - accuracy: 0.7353 - loss: 0.6540 - val_accuracy: 0.7284 - val_loss: 0.6513

Epoch 16/25
41/41  533s 13s/step - accuracy: 0.7353 - loss: 0.6495 - val_accuracy: 0.7284 - val_loss: 0.6498

Epoch 17/25
41/41  558s 13s/step - accuracy: 0.7477 - loss: 0.6421 - val_accuracy: 0.7346 - val_loss: 0.6410

Epoch 18/25
41/41  533s 13s/step - accuracy: 0.7415 - loss: 0.6454 - val_accuracy: 0.7407 - val_loss: 0.6361

Epoch 19/25
41/41  530s 13s/step - accuracy: 0.7074 - loss: 0.6677 - val_accuracy: 0.7407 - val_loss: 0.6396

Epoch 20/25
41/41  531s 13s/step - accuracy: 0.7461 - loss: 0.6451 - val_accuracy: 0.7469 - val_loss: 0.6343

Epoch 21/25
41/41  556s 14s/step - accuracy: 0.7461 - loss: 0.6322 - val_accuracy: 0.7407 - val_loss: 0.6338

Epoch 22/25
41/41  549s 13s/step - accuracy: 0.7337 - loss: 0.6393 - val_accuracy: 0.7469 - val_loss: 0.6318

Epoch 23/25
41/41  532s 13s/step - accuracy: 0.7601 - loss: 0.6296 - val_accuracy: 0.7531 - val_loss: 0.6269

Epoch 24/25
41/41  530s 13s/step - accuracy: 0.7415 - loss: 0.6411 - val_accuracy: 0.7593 - val_loss: 0.6195

Epoch 25/25
41/41  531s 13s/step - accuracy: 0.7724 - loss: 0.6248 - val_accuracy: 0.7963 - val_loss: 0.6121

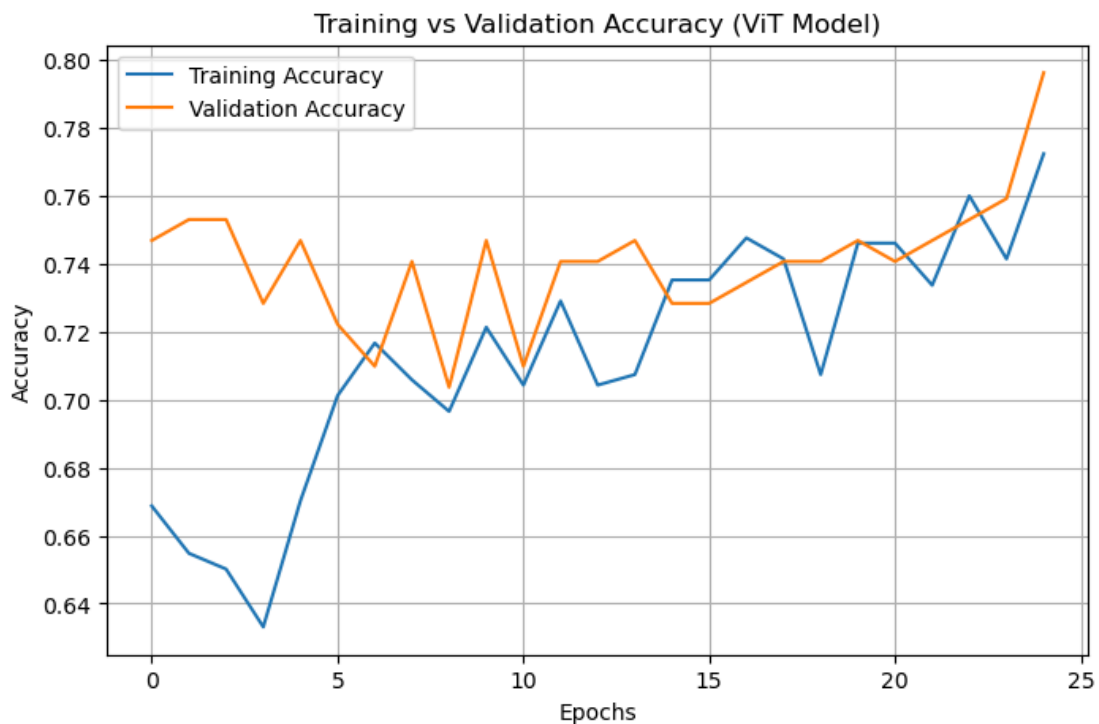
```
In [22]: model.save("ViT_BASELINE_FROZEN_80ACC.keras")
print("Baseline (~80%) model saved")
```

Baseline (~80%) model saved

```
In [23]: import matplotlib.pyplot as plt

train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

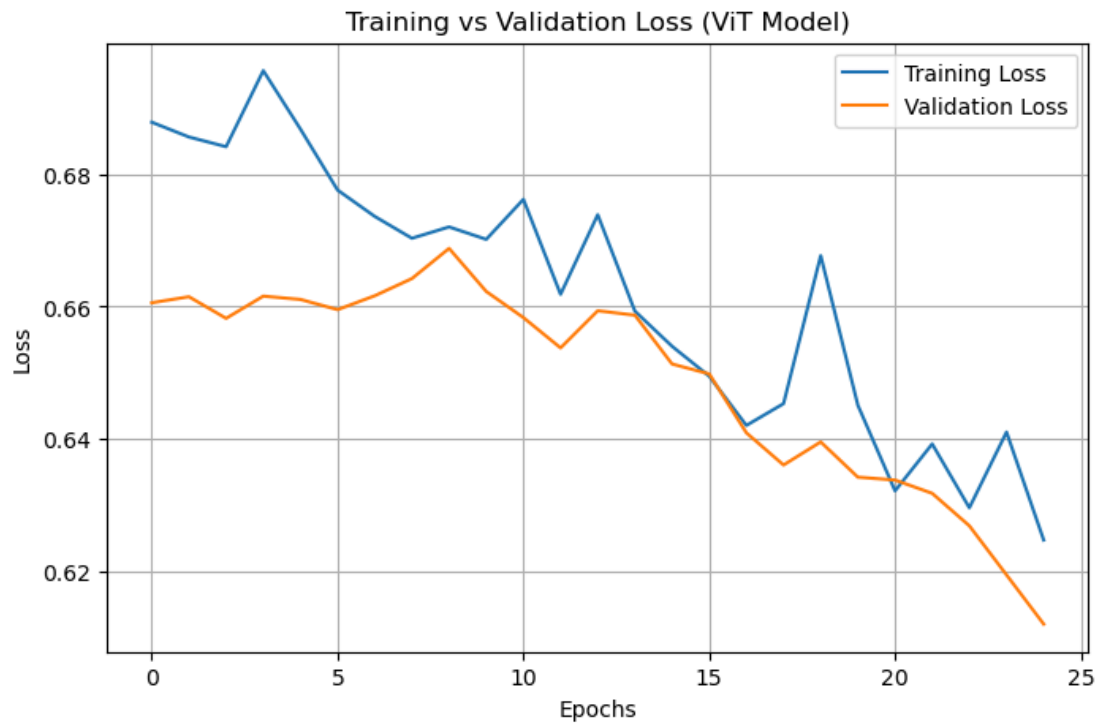
plt.figure(figsize=(8,5))
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy (ViT Model)')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [24]: import matplotlib.pyplot as plt

train_loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure(figsize=(8,5))
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss (ViT Model)')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [25]: import numpy as np

y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int).ravel()
```

6/6 ————— 108s 17s/step

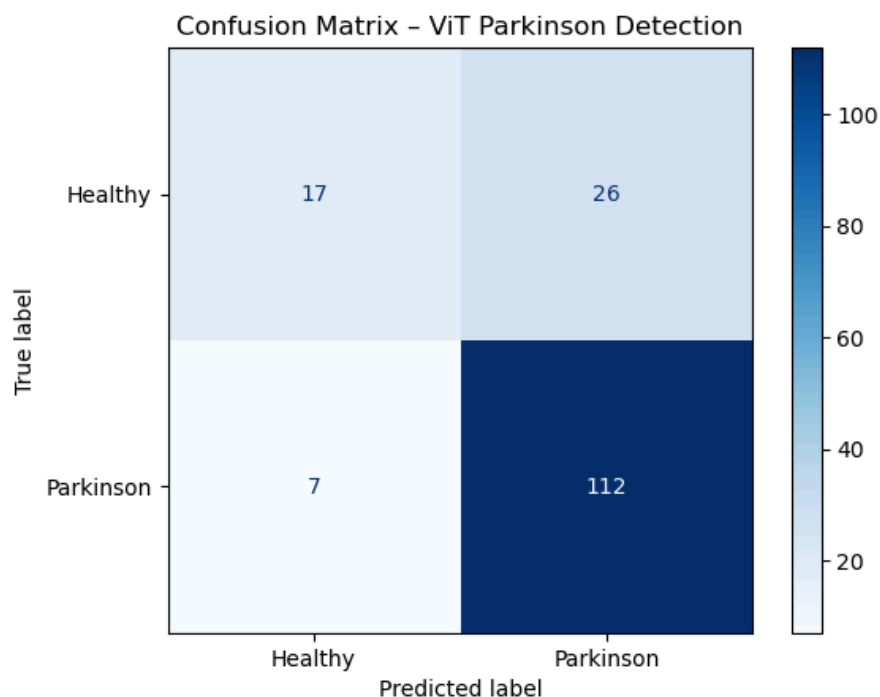
```
In [26]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Healthy", "Parkinson"]
)

plt.figure(figsize=(5,5))
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - ViT Parkinson Detection")
plt.show()
```

<Figure size 500x500 with 0 Axes>



```
In [27]: from sklearn.metrics import classification_report
```

```
print(classification_report(
    y_test,
    y_pred,
    target_names=["Healthy", "Parkinson"]
))
```

	precision	recall	f1-score	support
Healthy	0.71	0.40	0.51	43
Parkinson	0.81	0.94	0.87	119
accuracy			0.80	162
macro avg	0.76	0.67	0.69	162
weighted avg	0.78	0.80	0.77	162

```
In [28]: #phase 2 ( unfreezing last vit layer and softening data augmentation)
```

```
In [31]: from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
In [32]: datagen_soft = ImageDataGenerator(
    rotation_range=5,
    zoom_range=0.05,
    width_shift_range=0.05,
    height_shift_range=0.05,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
In [33]: # Unfreeze last ViT layers
for layer in vit_base.layers[-20:]:
    layer.trainable = True
```

```
In [34]: model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
In [36]: from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

```
In [37]: early_stop = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True,
```

```

    verbose=1
)

```

```

In [38]: checkpoint = ModelCheckpoint(
    filepath="VIT_PHASE2_BEST_EPOCH.keras",
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=False,
    verbose=1
)

```

```

In [39]: history_phase2 = model.fit(
    datagen_soft.flow(X_train, y_train, batch_size=8),
    validation_data=(X_test, y_test),
    epochs=6,
    class_weight=class_weights_dict,
    callbacks=[early_stop, checkpoint],
    verbose=1
)

```

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```

    self._warn_if_super_not_called()
Epoch 1/6
81/81 _____ 0s 17s/step - accuracy: 0.7586 - loss: 0.6248
Epoch 1: val_loss improved from None to 0.61045, saving model to VIT_PHASE2_BEST_EPOCH.keras
81/81 _____ 1583s 19s/step - accuracy: 0.7399 - loss: 0.6403 - val_accuracy: 0.8025 - val_loss: 0.6105
Epoch 2/6
81/81 _____ 0s 17s/step - accuracy: 0.7312 - loss: 0.6395
Epoch 2: val_loss improved from 0.61045 to 0.60331, saving model to VIT_PHASE2_BEST_EPOCH.keras
81/81 _____ 1468s 18s/step - accuracy: 0.7291 - loss: 0.6380 - val_accuracy: 0.7654 - val_loss: 0.6033
Epoch 3/6
81/81 _____ 0s 17s/step - accuracy: 0.7078 - loss: 0.6350
Epoch 3: val_loss improved from 0.60331 to 0.60302, saving model to VIT_PHASE2_BEST_EPOCH.keras
81/81 _____ 1516s 19s/step - accuracy: 0.7353 - loss: 0.6280 - val_accuracy: 0.8148 - val_loss: 0.6030
Epoch 4/6
81/81 _____ 0s 17s/step - accuracy: 0.7640 - loss: 0.6305
Epoch 4: val_loss did not improve from 0.60302
81/81 _____ 1455s 18s/step - accuracy: 0.7167 - loss: 0.6461 - val_accuracy: 0.7531 - val_loss: 0.6377
Epoch 5/6
81/81 _____ 0s 16s/step - accuracy: 0.7056 - loss: 0.6651
Epoch 5: val_loss did not improve from 0.60302
81/81 _____ 1388s 17s/step - accuracy: 0.7276 - loss: 0.6448 - val_accuracy: 0.8210 - val_loss: 0.6301
Epoch 6/6
81/81 _____ 0s 16s/step - accuracy: 0.7233 - loss: 0.6427
Epoch 6: val_loss improved from 0.60302 to 0.60207, saving model to VIT_PHASE2_BEST_EPOCH.keras
81/81 _____ 1414s 17s/step - accuracy: 0.7322 - loss: 0.6408 - val_accuracy: 0.7531 - val_loss: 0.6021
Restoring model weights from the end of the best epoch: 6.

```

```

In [41]: from tensorflow.keras.models import load_model

model = load_model(
    "VIT_PHASE2_BEST_EPOCH.keras",
    safe_mode=False
)

print("Best Phase-2 model loaded successfully")

```

Best Phase-2 model loaded successfully

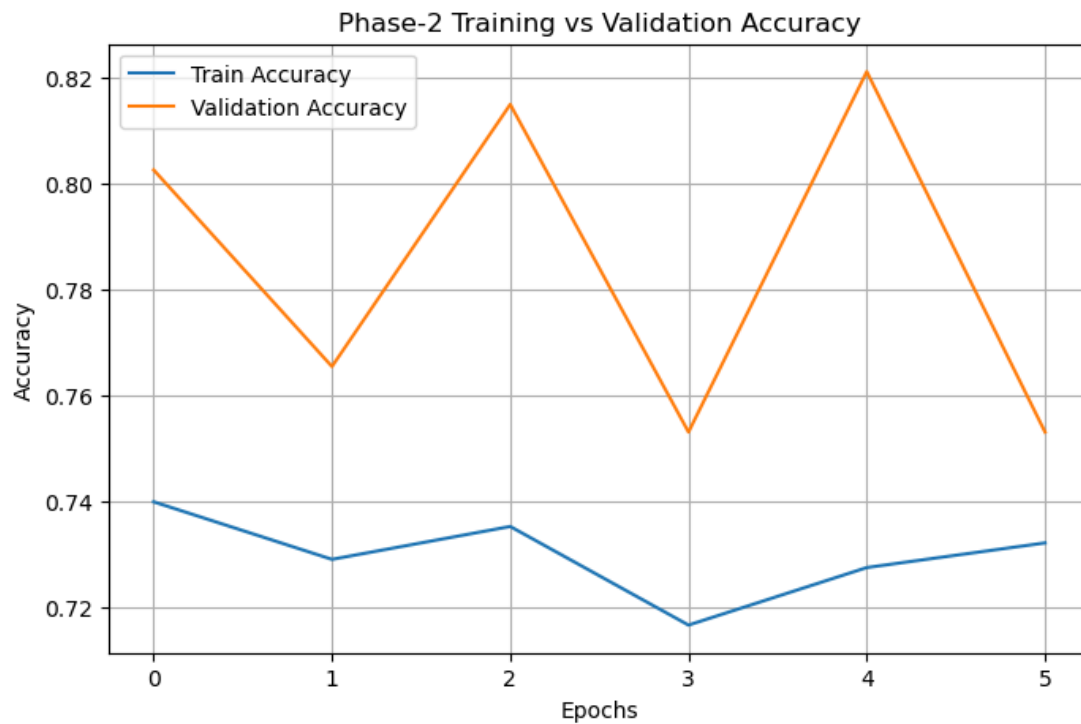
```

In [42]: import matplotlib.pyplot as plt

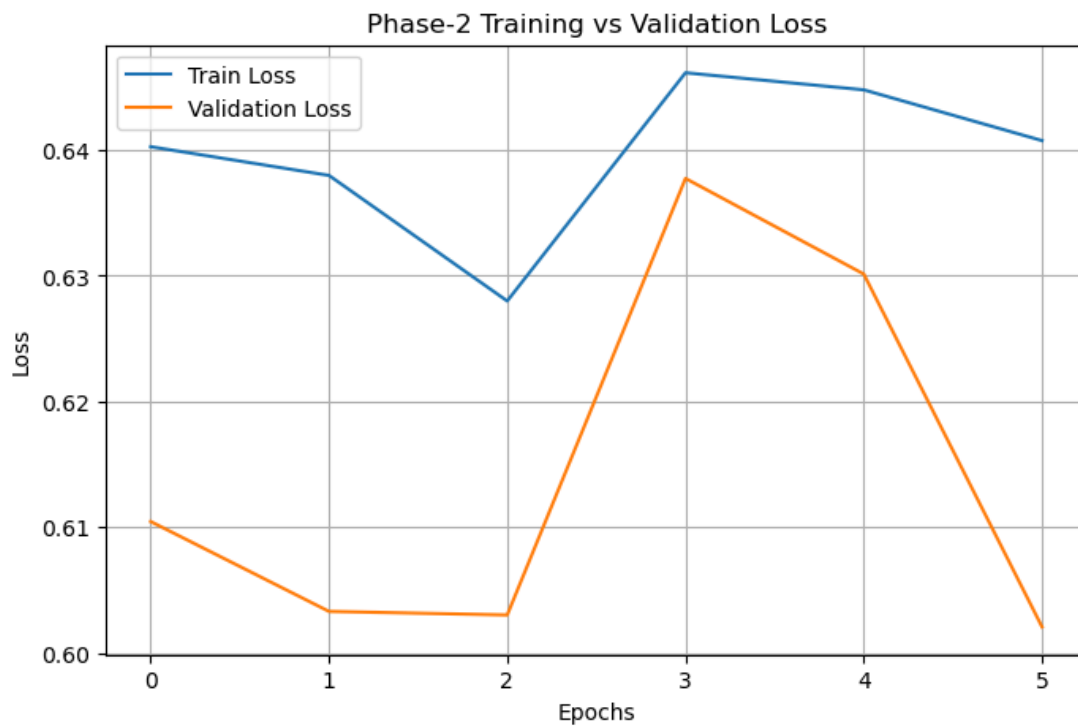
plt.figure(figsize=(8,5))
plt.plot(history_phase2.history['accuracy'], label='Train Accuracy')
plt.plot(history_phase2.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Phase-2 Training vs Validation Accuracy')

```

```
plt.legend()
plt.grid(True)
plt.show()
```



```
In [43]: plt.figure(figsize=(8,5))
plt.plot(history_phase2.history['loss'], label='Train Loss')
plt.plot(history_phase2.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Phase-2 Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [44]: import numpy as np

y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int).ravel()
```

6/6 ————— 118s 18s/step

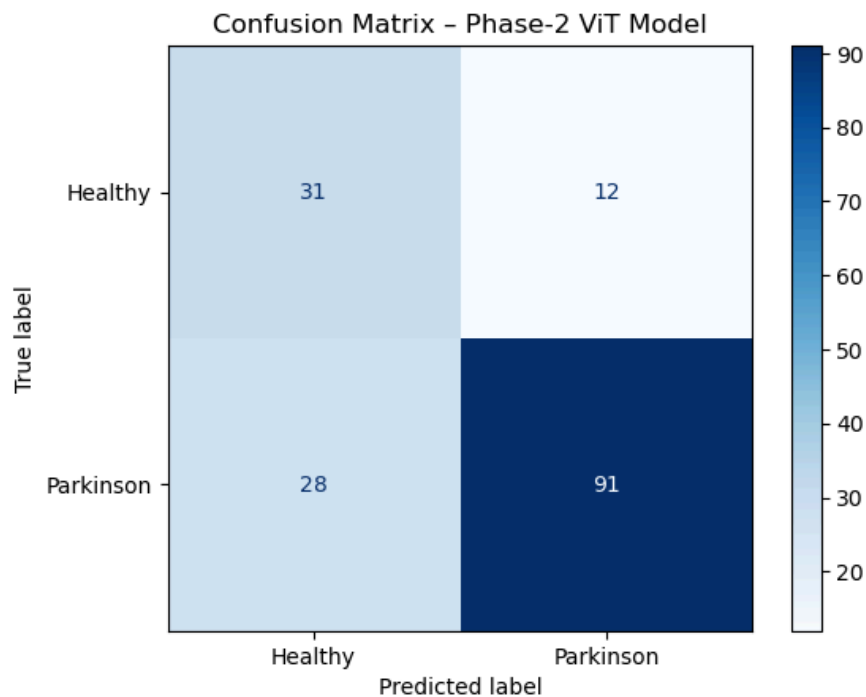
```
In [45]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Healthy", "Parkinson"]
)

plt.figure(figsize=(5,5))
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Phase-2 ViT Model")
plt.show()
```

<Figure size 500x500 with 0 Axes>



```
In [46]: from sklearn.metrics import classification_report

print(classification_report(
    y_test,
    y_pred,
    target_names=["Healthy", "Parkinson"]
))
```

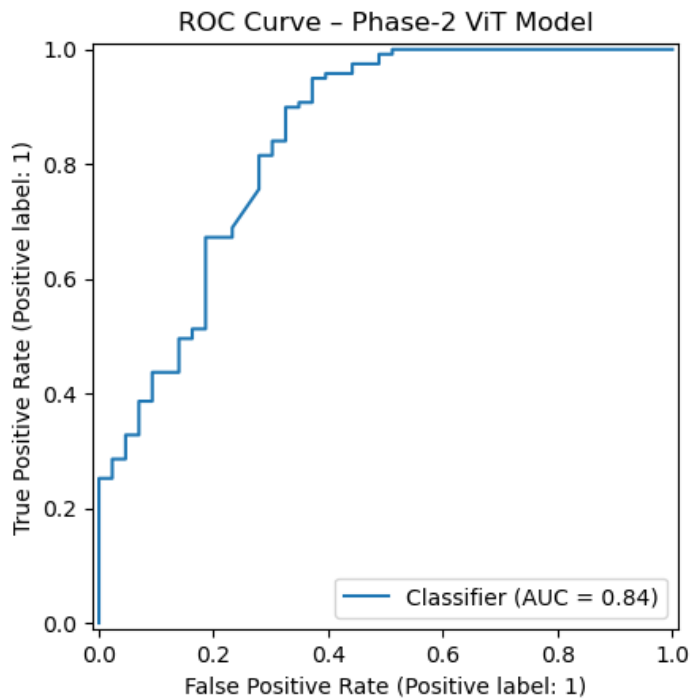
	precision	recall	f1-score	support
Healthy	0.53	0.72	0.61	43
Parkinson	0.88	0.76	0.82	119
accuracy			0.75	162
macro avg	0.70	0.74	0.71	162
weighted avg	0.79	0.75	0.76	162

```
In [47]: from sklearn.metrics import roc_auc_score, RocCurveDisplay

roc_auc = roc_auc_score(y_test, y_pred_prob)
print("ROC-AUC:", roc_auc)

RocCurveDisplay.from_predictions(y_test, y_pred_prob)
plt.title("ROC Curve - Phase-2 ViT Model")
plt.show()
```

ROC-AUC: 0.841704123509869



In [48]: `#Phase 3`

In [62]: `#THRESHOLD TUNING`

```
In [63]: from tensorflow.keras.models import load_model

model = load_model(
    "ViT_PHASE2_BEST_EPOCH.keras",
    safe_mode=False
)

print("Phase-2 best model loaded for threshold tuning")
```

Phase-2 best model loaded for threshold tuning

```
In [64]: import numpy as np

y_prob = model.predict(X_test).ravel()
```

WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x00000200A7A01630> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

6/6 ————— 110s 17s/step

```
In [65]: from sklearn.metrics import accuracy_score, recall_score

thresholds = np.arange(0.3, 0.71, 0.05)

print("Threshold | Accuracy | Healthy Recall | Parkinson Recall")
print("-"*55)

for t in thresholds:
    y_pred = (y_prob > t).astype(int)

    acc = accuracy_score(y_test, y_pred)
    rec_healthy = recall_score(y_test, y_pred, pos_label=0)
    rec_pd = recall_score(y_test, y_pred, pos_label=1)

    print(f"{t:.2f} | {acc:.3f} | {rec_healthy:.3f} | {rec_pd:.3f}")
```

Threshold	Accuracy	Healthy Recall	Parkinson Recall
0.30	0.858	0.535	0.975
0.35	0.858	0.581	0.958
0.40	0.864	0.628	0.950
0.45	0.827	0.674	0.882
0.50	0.753	0.721	0.765
0.55	0.481	0.953	0.311
0.60	0.309	1.000	0.059
0.65	0.272	1.000	0.008
0.70	0.265	1.000	0.000

In [66]: `best_threshold = 0.6`

```
y_pred_final = (y_prob > best_threshold).astype(int)
```

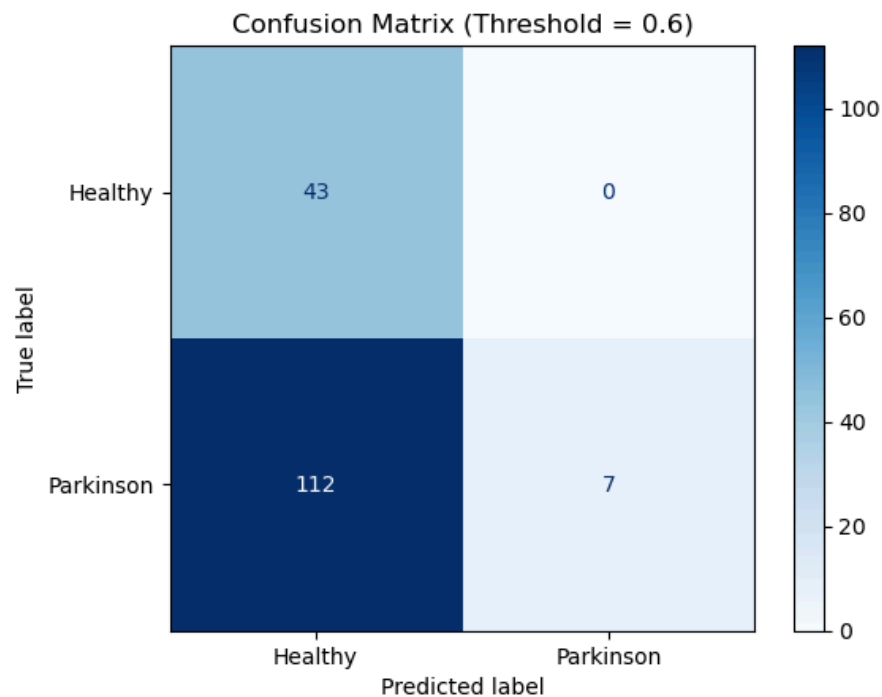
In [67]: `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay`
`import matplotlib.pyplot as plt`

```
cm = confusion_matrix(y_test, y_pred_final)
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Healthy", "Parkinson"]
)
```

```
plt.figure(figsize=(5,5))
disp.plot(cmap="Blues", values_format="d")
plt.title(f"Confusion Matrix (Threshold = {best_threshold})")
plt.show()
```

<Figure size 500x500 with 0 Axes>



In [68]: `from sklearn.metrics import classification_report`

```
print(classification_report(
    y_test,
    y_pred_final,
    target_names=["Healthy", "Parkinson"]
))
```

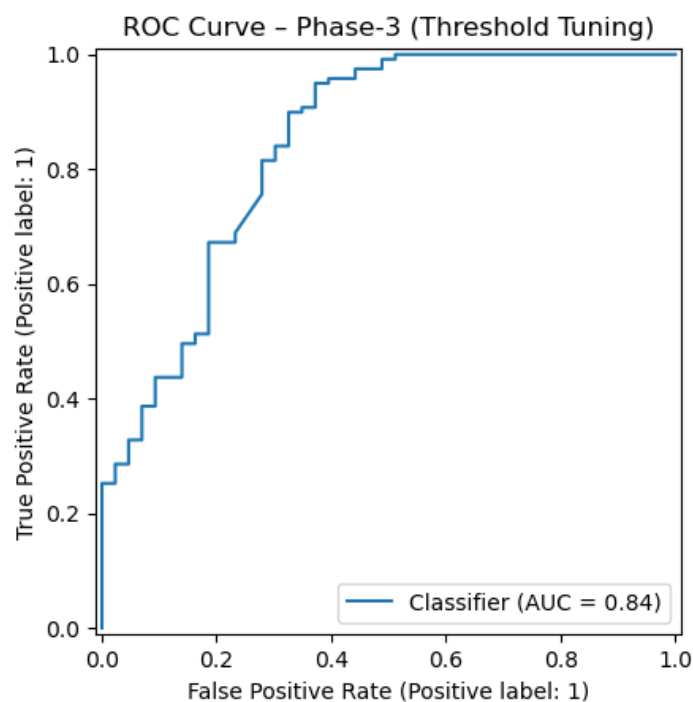
	precision	recall	f1-score	support
Healthy	0.28	1.00	0.43	43
Parkinson	1.00	0.06	0.11	119
accuracy			0.31	162
macro avg	0.64	0.53	0.27	162
weighted avg	0.81	0.31	0.20	162

```
In [69]: from sklearn.metrics import roc_auc_score, RocCurveDisplay

roc_auc = roc_auc_score(y_test, y_prob)
print("ROC-AUC:", roc_auc)

RocCurveDisplay.from_predictions(y_test, y_prob)
plt.title("ROC Curve - Phase-3 (Threshold Tuning)")
plt.show()
```

ROC-AUC: 0.841704123509869



```
In [84]: import pandas as pd
import numpy as np
from sklearn.metrics import (
    accuracy_score,
    recall_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)
```

```
In [87]: from tensorflow.keras.models import load_model

model_phase1 = load_model(
    "VIT_BASELINE_FROZEN_80ACC.keras",
    safe_mode=False
)

print(" Phase-1 (Baseline) model loaded")
```

Phase-1 (Baseline) model loaded

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\saving\saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 458 variables whereas the saved optimizer has 74 variables.

```
saveable.load_own_variables(weights_store.get(inner_path))
```

```
In [88]: y_prob_p1 = model_phase1.predict(X_test).ravel()
y_pred_p1 = (y_prob_p1 > 0.5).astype(int)
```

WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000001FF8CE2F880> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

6/6 ————— 106s 16s/step

```
In [89]: EXCEL_FILE_NAME = "Final Parkinson's Custom ViT Result .xlsx"
        FINAL_THRESHOLD = 0.40
```

```
In [90]: phase1_metrics = pd.DataFrame({
        "Metric": ["Accuracy", "Healthy Recall", "Parkinson Recall", "ROC-AUC"],
        "Value": [
            accuracy_score(y_test, y_pred_p1),
            recall_score(y_test, y_pred_p1, pos_label=0),
            recall_score(y_test, y_pred_p1, pos_label=1),
            roc_auc_score(y_test, y_prob_p1)
        ]
    })
```

```
In [91]: phase1_report = pd.DataFrame(
        classification_report(
            y_test, y_pred_p1,
            target_names=["Healthy", "Parkinson"],
            output_dict=True
        )
    ).transpose()
```

```
In [92]: phase1_cm = pd.DataFrame(
        confusion_matrix(y_test, y_pred_p1),
        index=["Actual Healthy", "Actual Parkinson"],
        columns=["Predicted Healthy", "Predicted Parkinson"]
    )
```

```
In [94]: phase1_history = pd.DataFrame({
        "Note": [
            "Phase-1 training history not stored.",
            "Phase-1 used as baseline feature extractor.",
            "Detailed epoch-wise history available for Phase-2 and Phase-3."
        ]
    })
```

```
In [96]: pip install openpyxl
```

```
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
```

```
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 1/2 [openpyxl]
----- 2/2 [openpyxl]
```

Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5

Note: you may need to restart the kernel to use updated packages.

```
In [97]: y_prob_p2 = model.predict(X_test).ravel() # Phase-2 best model
        y_pred_p2 = (y_prob_p2 > 0.5).astype(int)
```

6/6 ————— 127s 17s/step

```
In [98]: phase2_metrics = pd.DataFrame({
        "Metric": ["Accuracy", "Healthy Recall", "Parkinson Recall", "ROC-AUC"],
```

```

    "Value": [
        accuracy_score(y_test, y_pred_p2),
        recall_score(y_test, y_pred_p2, pos_label=0),
        recall_score(y_test, y_pred_p2, pos_label=1),
        roc_auc_score(y_test, y_prob_p2)
    ]
})

```

```

In [99]: phase2_report = pd.DataFrame(
    classification_report(
        y_test, y_pred_p2,
        target_names=["Healthy", "Parkinson"],
        output_dict=True
    )
).transpose()

```

```

In [100]: phase2_cm = pd.DataFrame(
    confusion_matrix(y_test, y_pred_p2),
    index=["Actual Healthy", "Actual Parkinson"],
    columns=["Predicted Healthy", "Predicted Parkinson"]
)

```

```

In [101]: phase2_history = pd.DataFrame({
    "Epoch": range(1, len(history_phase2.history["accuracy"]) + 1),
    "Train Accuracy": history_phase2.history["accuracy"],
    "Val Accuracy": history_phase2.history["val_accuracy"],
    "Train Loss": history_phase2.history["loss"],
    "Val Loss": history_phase2.history["val_loss"]
})

```

```

In [102]: thresholds = np.arange(0.3, 0.71, 0.05)
rows = []

for t in thresholds:
    pred = (y_prob_p2 > t).astype(int)
    rows.append({
        "Threshold": round(t, 2),
        "Accuracy": accuracy_score(y_test, pred),
        "Healthy Recall": recall_score(y_test, pred, pos_label=0),
        "Parkinson Recall": recall_score(y_test, pred, pos_label=1)
    })

phase3_threshold_df = pd.DataFrame(rows)

```

```

In [103]: with pd.ExcelWriter(EXCEL_FILE_NAME, engine="openpyxl") as writer:
    phase1_metrics.to_excel(writer, sheet_name="Phase1_Metrics", index=False)
    phase1_report.to_excel(writer, sheet_name="Phase1_Classification_Report")
    phase1_cm.to_excel(writer, sheet_name="Phase1_Confusion_Matrix")
    phase1_history.to_excel(writer, sheet_name="Phase1_Training_History", index=False)

    phase2_metrics.to_excel(writer, sheet_name="Phase2_Metrics", index=False)
    phase2_report.to_excel(writer, sheet_name="Phase2_Classification_Report")
    phase2_cm.to_excel(writer, sheet_name="Phase2_Confusion_Matrix")
    phase2_history.to_excel(writer, sheet_name="Phase2_Training_History", index=False)

    phase3_threshold_df.to_excel(writer, sheet_name="Phase3_Threshold_Analysis", index=False)

    print(" ALL Phase 1, 2, 3 results saved successfully!")

```

ALL Phase 1, 2, 3 results saved successfully!

In []: